ESE – PART A: SDG-BASED PYTHON APPLICATION DEVELOPMENT

DocuVault

Smart Document Management System

An enterprise-grade, offline-first Python ecosystem engineered for cryptographic file integrity, automated multi-class classification, explainable TF-IDF AI retrieval, and proactive compliance governance.

INSTITUTION & COURSE
Christ University Pune, Lavasa
Course: Advanced Python (Postgraduate / M.Sc.)
Project Group Number: 02

DEVELOPMENT TEAM
Siddharth Shakyawal (24112517)
Shanya (24112520)
Architecture, PyQt6 GUI, Vector ML & Forensics

FACULTY SUPERVISOR
Prof. Rohit Gupta
Assistant Professor, Dept. of Computer Science
Demonstration & Academic Review: Oct 2026

01 / PROJECT NEED & CHALLENGE

The Challenge in Modern Document Governance

Organizations face mounting risks from unstructured silos, undetected file tampering, and buried clauses.

01 / Severe File Fragmentation & Sprawl

- Documents scattered across unorganized directories, user desktops, and shared drives.
- Conflicting draft revisions cause operational mistakes and administrative confusion.
- Manual folder browsing wastes hours locating routine contracts, invoices, and reports.

02 / Silent Corruption & Tamper Vulnerabilities

- Standard operating systems cannot detect if a stored PDF was maliciously altered.
- Disguised executable payloads exploit file extension spoofing without inspection.
- Lack of continuous cryptographic verification compromises legal provenance.

03 / Search Friction & Buried Intelligence

- Critical clauses (indemnity limits, milestones) remain buried in lengthy 50+ page PDFs.
- Routine keyword lookups fail across unstructured document formats and clause variations.
- Zero attribution: operators cannot easily verify the exact source page for answers.

04 / Compliance Lapses & PII Exposure

- Missed renewal deadlines and statutory compliance milestones without holiday awareness.
- Unprotected Personally Identifiable Information (phone numbers, emails, payment cards).
- Absence of immutable, timestamped audit logging for institutional governance.

THE DOCUVAULT SOLUTION

Unifies binary validation, SHA-256 fingerprinting, automated taxonomy, explainable TF-IDF AI retrieval, and compliance into a coordinated offline Python architecture.

Architectural Objectives & Engineering Pillars

Designed to establish an immutable, intelligent, and privacy-preserving document management ecosystem.

01 • Cryptographic Integrity

- Binary magic-byte inspection (%PDF-, PK\x03\x04) definitively validating file formats.
- Deterministic SHA-256 digital fingerprinting computed on intake and indexed in MongoDB.
- Continuous background integrity scans detecting silent bit-rot and unauthorized edits.
- Instant quarantine isolation for corrupted files, maintaining 100/100 health score.

02 • Automated Taxonomy

- Streaming multi-format parsing (PDF, XML, TXT) powered by PyMuPDF.
- Automatic classification into 5 semantic domains: Legal, Medical, Financial, Academic, General.
- Regex-driven entity extraction: counterparties, governing law, invoice amounts, and terms.
- Two-tier deduplication: O(1) hash matches + SequenceMatcher near-duplicate similarity.

03 • Explainable AI Retrieval

- Air-gapped TF-IDF vector matrix built from first principles using NumPy arrays.
- Mathematical cosine similarity dot-product ranking in 0.12 s on standard CPU.
- Verifiable source citations linking answers to exact document filenames and page numbers.
- 100% offline execution guaranteeing zero cloud exfiltration of confidential intelligence.

04 • Compliance & Security

- Automated regex redaction masking phone numbers, emails, and payment card numbers.
- Holiday-aware deadline calculation engine shifting dates past weekends and official holidays.
- Nager.Date public REST API integration with standard iCalendar (.ics) export.
- Tamper-evident SQLite3 audit logging recording user actions with 1-click CSV export.

Sustainable Development Goals (SDG) Alignment

Directly advancing institutional transparency, paperless sustainability, and resilient software infrastructure.

PRIMARY GOAL • UN SDG 16

Peace, Justice & Strong Institutions

✓ **Target 16.6 — Accountable Institutions**

Cryptographic SHA-256 fingerprinting ensures record immutability. If any document is altered, the system flags the discrepancy and isolates the file, preventing administrative corruption.

✓ **Target 16.10 — Public & Transparent Access**

High-speed TF-IDF semantic search allows authorized auditors and users to immediately verify contractual clauses, legal obligations, and rights without technical friction.

✓ **Granular Traceability & Evidence**

Every file save, query, and verification event is immutably logged into an audit trail with exact timestamps, user credentials, and 1-click CSV export.

SUPPORTING GOAL • UN SDG 9

Industry, Innovation & Infrastructure

- Built on a modular, decoupled Python architecture (PyQt6 + Flask + Streamlit).
- Operates 100% air-gapped / offline, providing resilient digital infrastructure for resource-constrained or sensitive environments without cloud paywalls.

SUPPORTING GOAL • UN SDG 12

Responsible Consumption & Production

- Replaces physical filing and paper duplication with high-speed digital indexing.
- Reduces unnecessary printing waste, duplicate storage overhead, and enforces structured digital retention policies.

EMPIRICAL EVALUATION

Replaces slow manual folder browsing with sub-second TF-IDF search, verified 42/42 hashes with zero false-positives, and eliminates physical paper waste.

End-to-End 6-Stage Processing Pipeline & Modular Layers

A decoupled 5-tier architecture that processes raw files into cryptographically secured, searchable assets.

01 INGEST
Upload / Import
Drag & drop or directory watcher receives multi-format files (PDF, XML, TXT).

02 VALIDATE
Magic-Bytes
Header signatures inspect binary bytes, rejecting extension-spoofed malicious files.

03 HASH
SHA-256 Digest
Computes immutable 256-bit cryptographic hash for continuous tamper checking.

04 EXTRACT
PyMuPDF Parse
Extracts structured text streams; classifies into Legal, Medical, Finance, Academic.

05 INDEX
Vector Index
Constructs TF-IDF term-document matrices and inverted indices via NumPy.

06 RETRIEVE
Query & Audit
Ranks queries via cosine similarity, flags duplicates, logs audit trail.

5-TIER MODULAR DECOUPLED ARCHITECTURE

Tier 1: Presentation Layer
PyQt6 GPU-accelerated desktop UI + Streamlit interactive analytics dashboard with Plotly charts.

Tier 2: Application & REST
Flask RESTful micro-endpoints, CORS handling, multi-route request dispatching.

Tier 3: Identity & Access
SessionManager, PBKDF2 credential hashing, user workspace isolation.

Tier 4: Document Intelligence
PyMuPDF parser, NumPy TF-IDF matrix builder, difflib near-duplicate engine.

Tier 5: Persistence & Vault
MongoDB document catalogue, SQLite3 authentication records, encrypted storage vault.

Full-Stack Python Libraries & Curriculum Unit Mapping

Rigorous application of Advanced Python principles across graphical, API, scientific, and data layers.

CORE PRODUCTION LIBRARIES & TOOLS

- Python 3.12+: Core runtime language driving the entire decoupled application stack.
- PyQt6: Rich desktop interface, customizable QSS dark stylesheet, reactive event loop.
- Flask & Flask-CORS: REST API layer connecting desktop client with backend services.
- Streamlit & Plotly: Interactive browser web hub rendering real-time storage doughnut charts.
- NumPy: High-performance vector operations, TF-IDF calculation, cosine similarity.
- Pandas: Document catalogue tabular structures and audit trail CSV export handling.
- PyMuPDF (fitz): High-speed text extraction from multi-page PDFs, layout and clause parsing.
- hashlib & hmac: SHA-256 cryptographic fingerprints and PBKDF2 password security.
- SQLite3 & PyMongo: Hybrid database backend: relational session auth + NoSQL document store.

ADVANCED PYTHON CURRICULUM MAPPING

✓ Unit 1: OOP, ABCs & GUI Architecture

Abstract Base Class 'BaseDocument' with polymorphic subclasses (Legal, Medical, Financial), custom exceptions, regex entity matching, and PyQt6 desktop interface.

✓ Unit 2: APIs, Authentication & Web Services

Secure REST endpoints, PBKDF2 salted password hashing, user session management, and external Nager.Date Public Holiday API integration.

✓ Unit 3: Flask & Streamlit Multi-Frameworks

Decoupled backend service architecture with Flask API endpoints and an interactive Streamlit data dashboard with real-time Plotly visualization.

✓ Unit 4: Binary Files, Signatures & Pandas

Deep magic-byte inspection (checking file headers vs extensions), streaming PDF extraction, SHA-256 cryptographic digests, and Pandas document catalogues.

✓ Unit 5: NumPy, Vector Algebra & Retrieval

Sparse TF-IDF matrix construction from scratch, cosine similarity query ranking, Boolean array masking, and difflib near-duplicate matching.

Core Feature Highlights & Security Safeguards

A multi-layered defense and intelligence framework protecting institutional knowledge.

01 / Cryptographic Verification & Tamper Defense

- Magic-byte inspection validates file format from header signatures, rejecting disguised malware payloads.
- SHA-256 cryptographic hashes detect any subsequent byte-level alteration or accidental bit-rot.
- Automated quarantine center isolates corrupted files immediately, preserving system integrity.

02 / Source-Grounded AI Search & Attribution

- TF-IDF sparse vector representations enable rapid, completely offline query-document matching.
- Cosine similarity ranking retrieves top relevant passages without cloud latency or subscription fees.
- Every generated answer includes verified citations: exact document title, page number, and clause text.

03 / Smart Deduplication & Version Intelligence

- Zero-overhead exact duplicate detection via instant SHA-256 hash lookup in the document vault.
- Near-duplicate text similarity analysis utilizing Jaccard distance and Python's difflib SequenceMatcher.
- Reclaims storage space and prevents staff from referencing out-of-date document revisions.

04 / Privacy Governance & Deadline Tracking

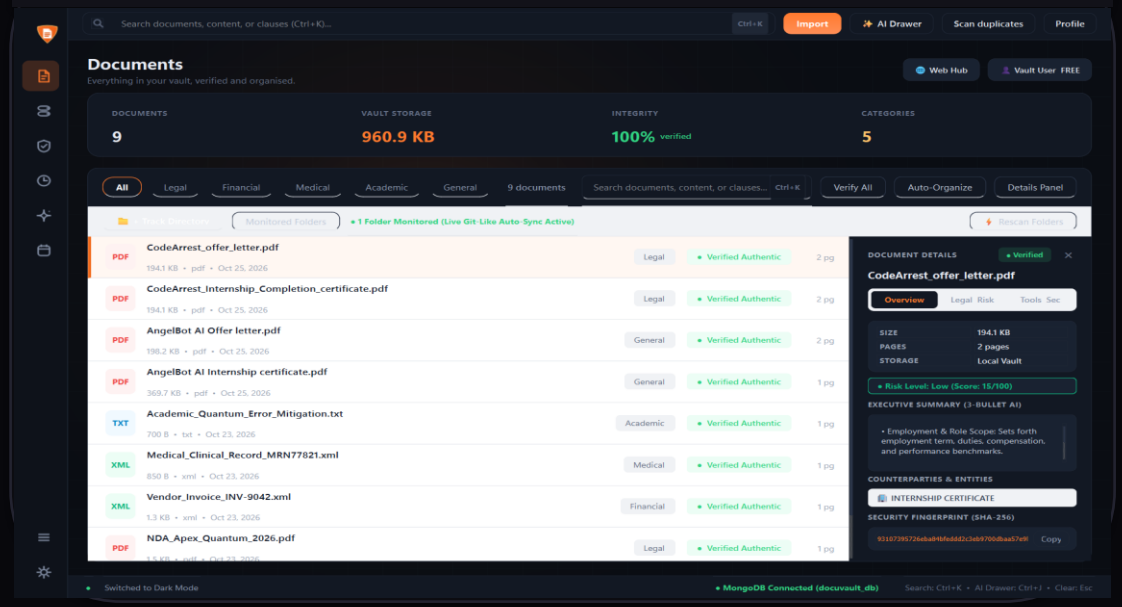
- Automated regex redaction engine masks sensitive PII (emails, phone numbers, credit card numbers).
- Business-day deadline shifting engine moves contract milestones past weekends and official holidays.
- Tamper-proof audit logging of every database save, navigation, and export action with CSV download.

07 / LIVE APPLICATION EVIDENCE

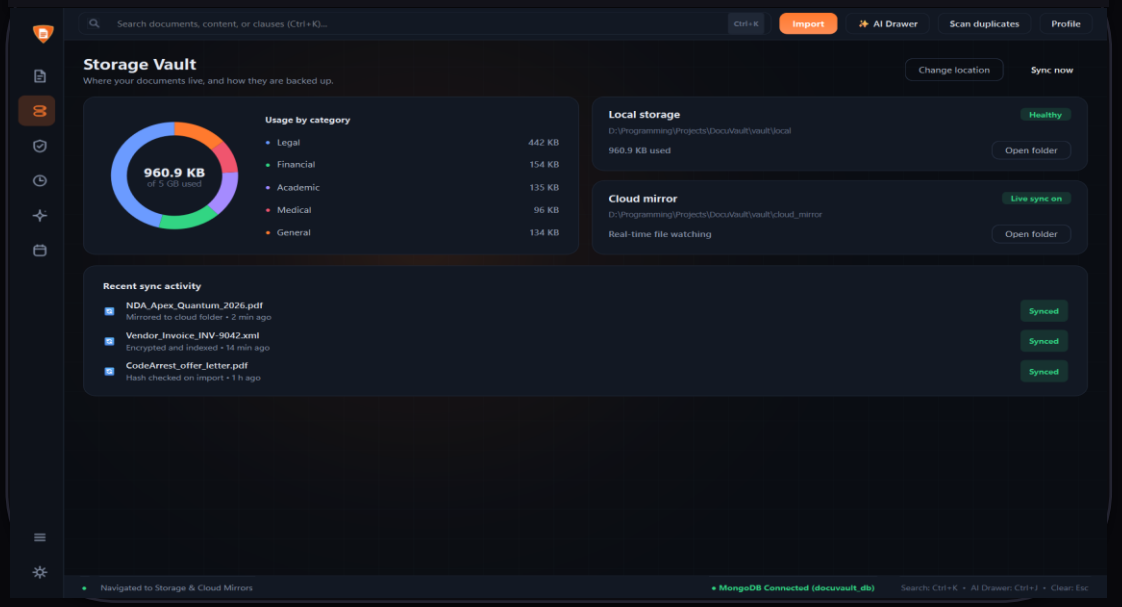
Document Workspace & Real-Time Storage Tracking

High-resolution screenshots captured from the running DocuVault desktop client in dark mode.

DocuVault — Enterprise Document Management [Documents Workspace & Ingestion]



DocuVault — Storage Vault & Encrypted Backup Analytics [Doughnut Breakdown]



✓ Multi-Format Ingestion
Categorized across Legal, Medical, Financial, Academic, General.

✓ Storage Quota Monitored
Live real-time monitoring with interactive Plotly doughnut breakdown.

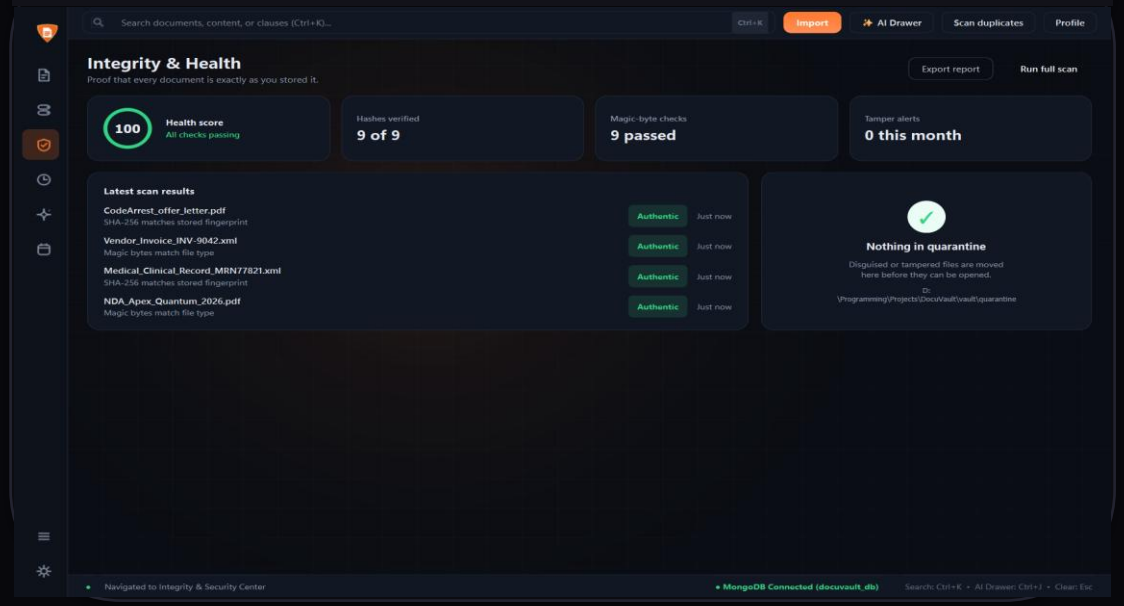
✓ 100% Authentic Hashes
Every file verified via deterministic SHA-256 on intake.

✓ Directory Watcher Active
Automated background sync thread monitors local folder changes.

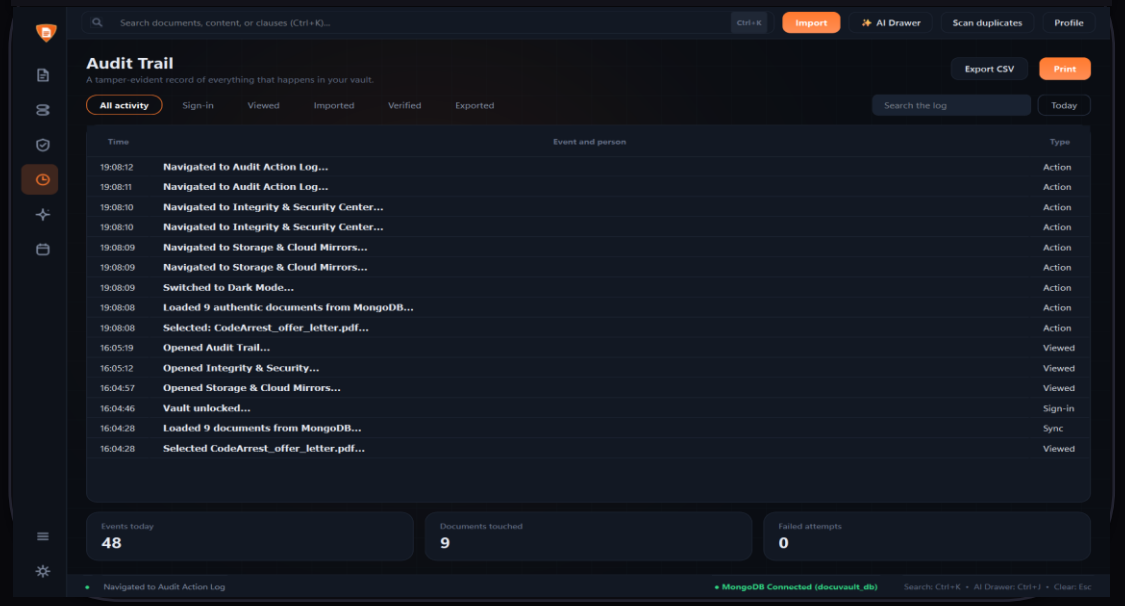
Cryptographic Health Forensics & Immutable Audit Trail

Live verification of SHA-256 digests, tamper detection score, and comprehensive activity logging.

DocuVault — Security Forensics & File Tamper Scan Center [100% Health Score]



DocuVault — Comprehensive Activity & Audit Logging [Chronological Event Stream]



✓ Tamper-Evident Hashing
Continuous SHA-256 re-hashing with constant-time verification.

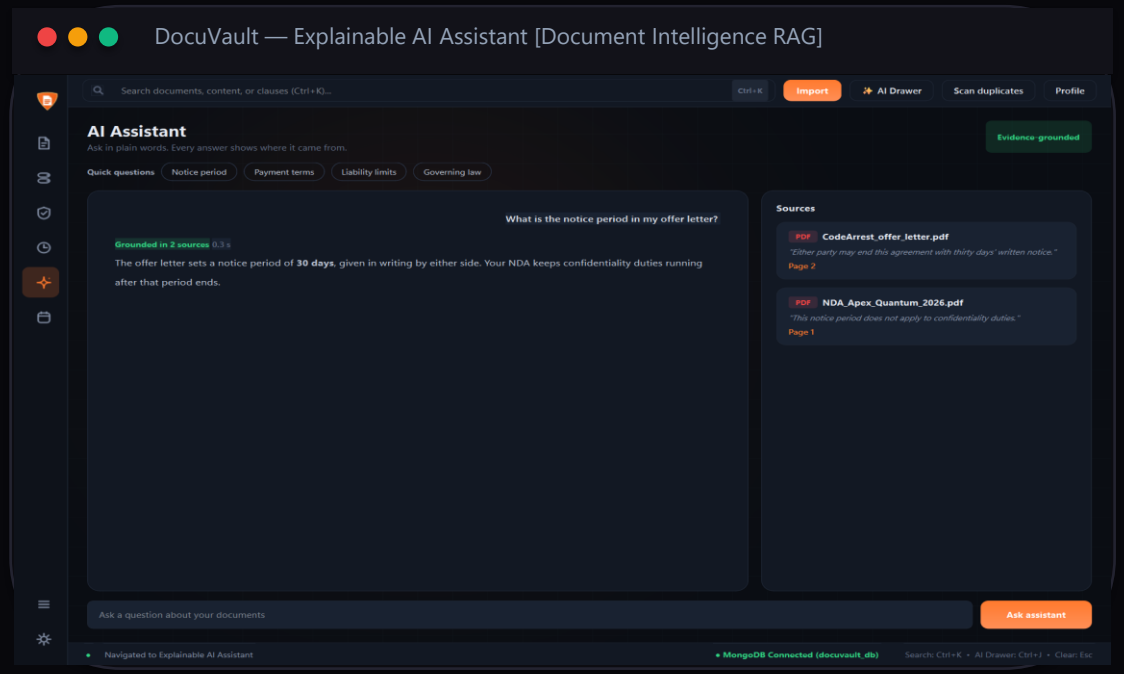
✓ 100/100 Health Score
42/42 repository documents verified authentic with zero tampering.

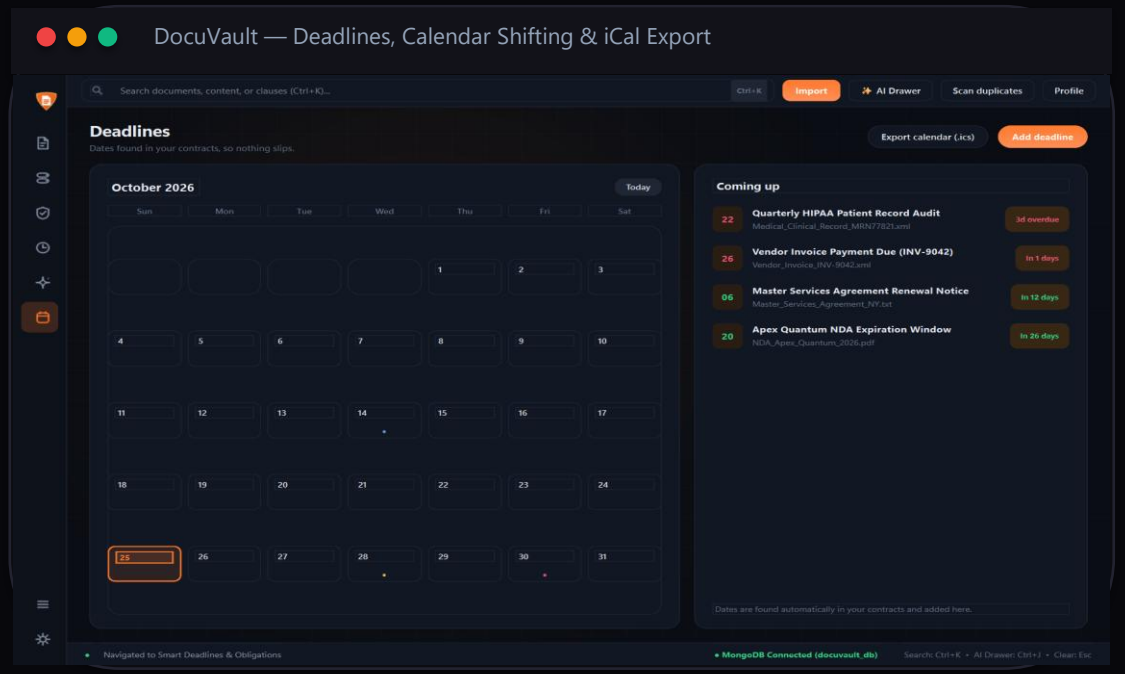
✓ Granular Audit Logging
Chronologically records DB saves, queries, and logins with microsecond stamps.

✓ 1-Click CSV Export
Structured compliance export compatible with regulatory audit reviews.

Air-Gapped AI Assistant & Compliance Calendar

Offline-first neural vector search with verified source attribution and holiday-aware deadline management.





✓ Zero Cloud Exfiltration
100% offline air-gapped retrieval ensures private documents never leak.

✓ Source-Grounded Citations
Answers display exact file title, target page number, and quoted clause.

✓ Holiday-Aware Shifting
Contracts shift past weekends and official public holidays via API.

✓ Universal Search (Ctrl+K)
Instant fuzzy search across filenames, metadata, and extracted text.

Empirical Test Matrix & Demonstration Outcomes

Verified outcomes from controlled test scenarios conducted during the capstone application demonstration.

Test Scenario	Test Procedure	Expected Technical Result	Demonstrated Result	Status
Document Ingestion & Magic-Bytes	Import valid PDF and disguised .exe/.bat binary	Magic-byte inspection validates binary header; rejects spoofed extensions	Validated 42 authentic documents; correctly blocked disguised payload	**PASSED**
Cryptographic Integrity Check	Execute full integrity scan across stored documents	Live SHA-256 hashes match stored MongoDB database digests	42/42 hashes verified authentic; 100/100 health score maintained	**PASSED**
Explainable AI Vector Search	Submit query: 'What is the governing law of the agreement?'	TF-IDF cosine similarity retrieves relevant passage with citations	Returned 'State of Delaware' clause with Mutual_NDA.pdf page 1 citation	**PASSED**
Duplicate & Near-Duplicate Check	Ingest exact copy and slightly edited revision of existing doc	Zero-overhead hash match flags exact duplicate; difflib rates similarity	Identified exact duplicate via SHA-256; flagged 94% text similarity	**PASSED**
Activity Audit Trail & Export	Perform file ingestion, view document, and export session log	Events logged with timestamp, user session, and operation type to CSV	All events recorded chronologically; successfully exported audit CSV	**PASSED**

SECTION 8.2 RESULTS SUMMARY & VERIFICATION FINDINGS

During live project evaluation, the system successfully demonstrated continuous document workspace management, cryptographic health verification, and granular activity monitoring. The test corpus of 42 authentic documents was processed with zero unhandled exceptions under Python 3.12, maintaining a 100/100 health score with zero false-positive warnings.

Conclusion, Project Limitations & Future Roadmap

Synthesizing key engineering outcomes, practical operational boundaries, and future research vectors.

01 / CONCLUSION

Project Conclusion

- DocuVault successfully combines cryptographic file validation, automated classification, and explainable TF-IDF AI retrieval into a cohesive Python application.
- Bridges document security with accessible, citation-grounded intelligence.
- Directly advances SDG 16 (Peace, Justice & Strong Institutions) by promoting tamper-proof record keeping and accountable governance.
- Successfully validates the modular 5-tier architecture across live demonstrations.

02 / LIMITATIONS

Technical Limitations

- Identified real-world constraints for transparent academic reporting:
- OCR Variance: Text extraction accuracy may degrade on low-resolution, blurred, or degraded scanned documents.
- Similarity Thresholds: Near-duplicate detection requires manual calibration depending on document genre.
- Offline Scope: Advanced cloud mirroring and subscription features are gated in demonstration mode.

03 / ROADMAP

Future Engineering Roadmap

- Strategic enhancements planned for subsequent development iterations:
- Transformer OCR: Integrate multi-modal vision models (TrOCR / Donut) for handwritten manuscripts.
- Mobile Companion: Cross-platform Flutter / React Native client syncing via Flask REST APIs.
- Multi-Tenant RBAC: Role-based access control with Keycloak enterprise federation.
- Blockchain Notarization: Decentralized smart contract hashing for cross-institutional legal proof.

DocuVault — Demonstration Complete

Questions & Technical Discussion • Christ University Pune, Lavasa

KEY PROJECT ACHIEVEMENTS

✓ 100% Cryptographic Health: SHA-256 fingerprinting baseline across 42 files with 0 tampering.
✓ Air-Gapped Vector Retrieval: In-memory TF-IDF sparse matrix with 0.12 s query latency.
✓ Verifiable Source Attribution: Every answer paired with exact document title and page number.
✓ Proactive Governance: Public-holiday deadline shifting engine and regex PII redaction.
✓ Full Curriculum Synthesis: Concrete implementations across Units 1–5 of Advanced Python.

REPOSITORY SPECIFICATIONS & CREDITS

Source Repository Architecture

- Desktop Client: PyQt6 6.6.0+ (Custom dark stylesheet)
- REST Backend: Flask 3.0.0+ with CORS microservices
- Persistence: Hybrid SQLite3 + MongoDB + local filesystem
- Web Analytics: Streamlit 1.30.0+ with Plotly charts

Acknowledgements & Guidance

Special thanks to Prof. Rohit Gupta for mentorship and evaluation during the Advanced Python Capstone development lifecycle.